Carpeta del Proyecto — ProgMesas

Desarrollo de un Sistema Web para Gestión de Programas de Examen Integrado con Google Drive (ProgMesas)

Alan Buzzatto y Valentino Rodriguez

Escuela de Educación Técnica N.° 2 "Independencia"

Proyecto de Práctica Profesional (PPP) — 7.° año, 1.ª división, Técnico en Computación

Profesor: Juan Roude

13 de noviembre de 2026

Índice

1. Presentación del proyecto..... 3
2. Objetivos..... 4
Objetivo general..... 4
Objetivos específicos..... 4
3. Descripción del dominio del sistema..... 5
Conceptos del dominio..... 5
Cursos de la escuela..... 5
Actores..... 6
Reglas del dominio..... 6
4. Límites del sistema..... 7
Marco legal..... 7
5. Requerimientos funcionales y no funcionales..... 9
Requerimientos funcionales..... 9
Requerimientos no funcionales..... 10
6. Diagrama de Gantt..... 12
7. Casos de uso..... 14
Descripción de CU07: Subir o reemplazar programa..... 15
Descripción de CU06: Vincular cuenta con la lista..... 16
8. Diagrama Entidad-Relación y diccionario de datos..... 17
Diccionario de datos..... 17
9. Conclusiones..... 21
10. Anexos..... 22
Anexo A: Relevamiento y estudio de factibilidad..... 22
Anexo B: Código..... 22
Anexo C: Documentación externa (guía de uso)..... 30
Anexo D: Documentación interna..... 32
Anexo E: Capturas de pantalla..... 38
11. Referencias..... 41

1. Presentación del proyecto

ProgMesas es un sistema web que centraliza los programas de examen de la Escuela Técnica N.° 2 "Independencia": los docentes los cargan en PDF, la asesoría pedagógica controla qué falta y cualquier alumno o familia los consulta sin iniciar sesión.

Antes de cada mesa de examen, la escuela necesita reunir un programa por cada materia y curso. Hasta ahora los archivos estaban dispersos, era difícil encontrar la versión actualizada, no había una organización por año, curso y materia, y no se podía seguir qué programas faltaban.

ProgMesas resuelve ese problema con tres piezas:

- **Carga ordenada**: cada docente puede subir el PDF del programa de cualquier curso y materia de la lista oficial de horarios (473 asignaciones); los suyos aparecen primero.
- **Control de entregas**: la asesoría tiene un tablero con el porcentaje de cobertura por curso, la fecha límite, el listado de docentes con programas pendientes y un registro de actividad.
- **Consulta pública**: alumnos y familias buscan por año, curso o materia, ven el PDF en el navegador y descargan todos los programas de un curso en un ZIP.

El sistema se integra con el campus virtual de la escuela: cada persona inicia sesión en el campus y entra a ProgMesas desde el módulo de programas de examen, sin otra contraseña. Además, ProgMesas copia el diseño del sistema de asistencia de la escuela (PROJECT_ASIST) para que la comunidad lo reconozca como parte de la misma plataforma. Los archivos se guardan en el Google Drive institucional.

El proyecto fue desarrollado por Alan Buzzatto y Valentino Rodriguez en el marco del Proyecto de Práctica Profesional de 7.° año. El código es público en GitHub (alan4352/progMesas) y cuenta con una demostración navegable en GitHub Pages.

En la escuela, ProgMesas funciona **dentro del campus virtual** (repositorio alan4352/CampusT2), como la carpeta `progmesas`: lo atiende `api.php`, en PHP con una base SQLite, en el mismo servidor que el campus, y los PDF se guardan en el Google Drive de la escuela. El servidor Node.js del repositorio propio queda como alternativa para usarlo de forma independiente; las dos versiones comparten la página, la base de datos y las reglas.

2. Objetivos

Objetivo general

Desarrollar un sistema web que permita a la Escuela Técnica N.° 2 "Independencia" reunir, controlar y publicar los programas de examen de todas las materias y cursos en un solo lugar, integrado al campus virtual de la escuela.

Objetivos específicos

1. Permitir que cada docente cargue, reemplace o reutilice en PDF el programa de cualquier curso y materia, sin depender de la aprobación manual de otra persona.
2. Dar a la asesoría pedagógica un tablero que muestre la cobertura por curso y por docente, los programas pendientes y atrasados respecto de una fecha límite.
3. Publicar los programas para que alumnos y familias los consulten sin cuenta, incluidos los de años anteriores, y descarguen los de un curso en un solo archivo.
4. Conservar el historial de versiones de cada programa y un registro de quién hizo cada cambio.
5. Guardar los archivos en el Google Drive de la escuela, con respaldo en el servidor cuando Drive no está disponible.
6. Usar el inicio de sesión del campus virtual de la escuela y el diseño de su sistema de asistencia (PROJECT_ASIST), para que los usuarios no tengan que aprender una herramienta distinta.
7. Asegurar la calidad con pruebas automáticas de la API y de la interfaz, incluida la accesibilidad (WCAG 2.1 AA).

3. Descripción del dominio del sistema

El dominio de ProgMesas es la gestión de los programas de examen de una escuela secundaria técnica: el documento que cada docente entrega por materia y curso, y que los alumnos usan para preparar las mesas de examen (regulares, previas y libres).

Conceptos del dominio

Concepto	Qué es en la escuela	Cómo lo representa ProgMesas
Programa de examen	Documento con los contenidos y la bibliografía que se evalúan en una materia	Un PDF asociado a una asignación y a un año lectivo; además del programa común, cada asignación puede tener un programa libre, para los alumnos que rinden en condición de libres
Asignación	Una materia dictada en un curso por uno o más docentes	Fila de la lista oficial (473 asignaciones), con id curso + materia, p. ej. `2-1_matematica`
Curso	Año y división (por ejemplo, 2.° 1.ª), con su turno	Campo de la asignación y filtro de búsqueda
Año lectivo	Ciclo escolar al que pertenece el programa	Campo del programa; permite consultar años anteriores
Fecha límite	Plazo que fija la asesoría para entregar los programas	Ajuste por año; marca atrasados y entregas fuera de término
Versión	Programa reemplazado por uno corregido	Historial con el PDF anterior, quién lo reemplazó y cuándo

Cursos de la escuela

La escuela tiene 29 cursos. Desde 4.° año, cada división corresponde a una especialidad (TC, TG o TAE).

Año	Cursos
1.°	1 1, 1 2, 1 3, 1 4, 1 5
2.°	2 1, 2 2, 2 3, 2 4
3.°	3 1, 3 2, 3 3
4.°	4 1 TC, 4 1 TG, 4 1 TAE, 4 2 TAE
5.°	5 1 TC, 5 1 TG, 5 1 TAE, 5 2 TAE
6.°	6 1 TC, 6 1 TG, 6 1 TAE
7.°	7 1 TC, 7 1 TG, 7 1 TAE

Actores

Actor	Rol en la escuela	Qué hace en el sistema
Alumno / familia / visitante	Consulta los programas para rendir	Busca, abre y descarga programas sin iniciar sesión
Docente	Dicta una o más materias	Carga, reemplaza y reutiliza los programas de sus asignaciones
Asesora pedagógica (en el campus, directivo)	Controla la entrega de programas	Usa el dashboard, fija la fecha límite, corrige vinculaciones, carga o elimina cualquier programa
Consulta (preceptor)	Supervisa	Ve el dashboard y el registro sin modificar nada (previsto: ProgMesas ya lo admite, pero el campus todavía no tiene el rol de preceptor)
Campus virtual de la escuela	Plataforma donde inician sesión alumnos, profesores y directivos	Arma un pase firmado con el usuario y su rol y abre ProgMesas
Google Drive	Almacenamiento institucional	Guarda los PDF en carpetas Año / Curso / Materia

Reglas del dominio

- Matemática de 2.° 1.ª y Matemática de 3.° 2.ª son programas distintos, aunque la materia se llame igual: el control se hace por asignación, no por nombre de materia.
- Cada asignación tiene un solo programa vigente por año, más un programa libre opcional; al reemplazar cualquiera de los dos, el anterior pasa al historial. Para la cobertura, la asignación cuenta como entregada con el programa común o con el libre.
- Un docente puede cargar el programa de cualquier asignación de la lista oficial; cada carga queda registrada con su cuenta y, si reemplaza un programa, la versión anterior se conserva.
- Las cuentas llegan verificadas desde el campus virtual: la cuenta se vincula sola con el nombre de la lista si coincide, o el docente lo elige (es opcional: sin vincular, figura con el nombre de su cuenta del campus). La asesora puede corregir o anular esa vinculación.
- Los programas de años anteriores siguen publicados, porque los alumnos que adeudan materias los necesitan.

4. Límites del sistema

ProgMesas cubre el ciclo de vida del programa de examen, desde la carga hasta la consulta; no gestiona usuarios, notas ni la organización de las mesas.

Dentro del alcance

- Carga, reemplazo, reutilización y eliminación de programas en PDF, comunes y libres.
- Control de cobertura por año, curso y docente, con fecha límite y recordatorios por correo (se abre el cliente de correo del usuario).
- Consulta pública, visor de PDF, enlaces por curso y descarga en ZIP.
- Historial de versiones, registro de actividad y respaldo de los datos.
- Almacenamiento en Google Drive con respaldo en el servidor.

Fuera del alcance

- Alta, baja y verificación de usuarios: las hace el campus virtual de la escuela. ProgMesas solo recibe el pase firmado que arma el campus.
- Armado de las mesas de examen: fechas, tribunales, inscripción de alumnos y calificaciones.
- Redacción o corrección del contenido de los programas: el sistema guarda el PDF que entrega el docente.
- Edición de la lista de horarios dentro de la página: la lista oficial se mantiene en una planilla Excel que la asesoría edita.
- Envío automático de correos o notificaciones desde el servidor.
- Formatos distintos de PDF.

Restricciones

- La instalación definitiva es dentro del campus virtual, en el servidor de la escuela (PHP con SQLite); Render se usa solo de forma temporal y GitHub Pages solo como demostración.
- Google Drive necesita la autorización de una cuenta de la escuela. Sin ella, los PDF quedan en el servidor.
- Cada PDF puede pesar hasta 10 MB (valor configurable).

Marco legal

Norma	Qué regula	Cómo la respeta ProgMesas
Ley 25.326 de Protección de los Datos Personales	Tratamiento de nombres, correos y otros datos personales	Guarda solo los datos necesarios (nombre, correo, rol), publica una política de privacidad y no permite cargar datos de alumnos en los programas.
Ley 11.723 de Propiedad Intelectual	Derechos de autor sobre obras, incluido el software y el material educativo	La autoría de cada programa sigue siendo de su docente; las condiciones del servicio lo establecen.
Ley 26.206 de Educación Nacional	Marco del sistema educativo	El sistema apoya un proceso institucional de la escuela sin reemplazar sus canales oficiales.

Norma	Qué regula	Cómo la respeta ProgMesas
Ley 26.653 de Accesibilidad de la Información en las Páginas Web	Accesibilidad de los sitios web	La interfaz cumple WCAG 2.1 AA, verificado con pruebas automáticas.

5. Requerimientos funcionales y no funcionales

El sistema cumple 18 requerimientos funcionales y 12 no funcionales; todos están implementados y la mayoría tiene pruebas automáticas (65 de la API, 5 de ellas sobre la versión PHP con un Google Drive simulado, y 12 de la interfaz).

Requerimientos funcionales

Código	Requerimiento	Actor
RF01	Iniciar sesión con el campus virtual: el campus arma un pase firmado y abre ProgMesas sin pedir otra contraseña (profesor → docente, directivo → asesora, alumno → alumno). ProgMesas no tiene login propio.	Todos los usuarios con cuenta
RF02	Vincular la cuenta del docente con su nombre en la lista oficial: automáticamente si coincide, o eligiéndolo (opcional: sirve para mostrar sus materias primero).	Docente
RF03	Mostrar al docente todas las asignaciones del año, con las suyas primero, el estado de cada programa y la fecha límite.	Docente
RF04	Subir el programa en PDF de cualquier asignación (curso y materia), para el año lectivo actual, indicando si es el programa libre; el título se genera solo y sólo aclara «libre» en ese caso.	Docente, Asesora
RF05	Reemplazar un programa conservando la versión anterior en el historial.	Docente, Asesora
RF06	Reutilizar el programa del año anterior para la misma asignación (común o libre) sin volver a subirlo.	Docente, Asesora
RF07	Eliminar un programa (el archivo se borra solo si ningún otro programa lo usa).	Asesora
RF08	Buscar programas por año, curso, materia o docente, incluidos los de años anteriores.	Alumno
RF09	Ver el PDF en un visor dentro de la página y descargarlo.	Alumno
RF10	Compartir el enlace de una búsqueda (filtros en la dirección de la página).	Alumno
RF11	Descargar en un ZIP todos los programas de un curso, ordenados por año.	Alumno

Código	Requerimiento	Actor
RF12	Mostrar el dashboard con cobertura del año, avance por curso y por docente, y pendientes.	Asesora, Consulta
RF13	Fijar la fecha límite de entrega de cada año y marcar atrasados y entregas fuera de término.	Asesora
RF14	Exportar pendientes a CSV, copiar los correos de docentes con pendientes y abrir un recordatorio por correo.	Asesora
RF15	Corregir o anular la vinculación de un docente y vincular programas viejos a una asignación.	Asesora
RF16	Registrar cada acción que modifica datos y mostrar el registro de actividad.	Sistema; Asesora y Consulta lo ven
RF17	Guardar los PDF en Google Drive y, si no está disponible, en el servidor, pasándolos a Drive automáticamente (al abrir el dashboard, como mucho cada 6 horas) o con un botón.	Sistema, Asesora
RF18	Respaldar la base de datos, los PDF que quedaron en el servidor y la planilla: los programas quedan en Google Drive y, con el servidor Node, hay además una copia automática diaria.	Sistema

Requerimientos no funcionales

Código	Tipo	Requerimiento
RNF01	Seguridad	Verificar la firma (HS256 con una clave compartida), el vencimiento (8 horas) y el rol del pase del campus virtual.
RNF02	Seguridad	No guardar usuarios ni contraseñas: el inicio de sesión lo hace el campus y ProgMesas sólo verifica el pase.
RNF03	Seguridad	Aceptar solo PDF válidos (se revisa el contenido) de hasta 10 MB.
RNF04	Seguridad	Escapar todo el contenido cargado por usuarios y bloquear scripts externos con una política CSP.
RNF05	Seguridad	Mantener las credenciales fuera del repositorio y pasarlas por variables de entorno.
RNF06	Usabilidad	Usar el mismo diseño que el sistema de asistencia de la escuela (PROJECT_ASIST), con modo claro y oscuro.
RNF07	Accesibilidad	Cumplir WCAG 2.1 AA en los dos modos, verificado con axe-core.

Código	Tipo	Requerimiento
RNF08	Portabilidad	Funcionar en celulares, tabletas y computadoras (diseño adaptable).
RNF09	Disponibilidad	Seguir funcionando si Google Drive no responde, guardando los PDF en el servidor.
RNF10	Mantenibilidad	Configurar el sistema con variables de entorno, sin tocar el código.
RNF11	Calidad	Ejecutar las pruebas automáticas en cada cambio con GitHub Actions (Node 20 y 22).
RNF12	Privacidad	Publicar la política de privacidad y las condiciones del servicio, y usar Drive solo para los fines declarados.

6. Diagrama de Gantt

El proyecto se realizó en 13 actividades entre el 23 de abril y el 13 de noviembre de 2026, día de entrega de la carpeta (la primera versión de la documentación se entregó el 27 de octubre; la actividad 13 suma la integración al campus de la escuela). Cada actividad depende de la anterior: empieza cuando la otra termina (las flechas). La 11 y la 12 se superponen del 23 al 26 de octubre, los únicos días con dos actividades a la vez; la franja de carga de trabajo muestra cuántas actividades hay en curso cada día. Octubre y noviembre se dibujan ampliados por semana para que se lean sus actividades cortas.

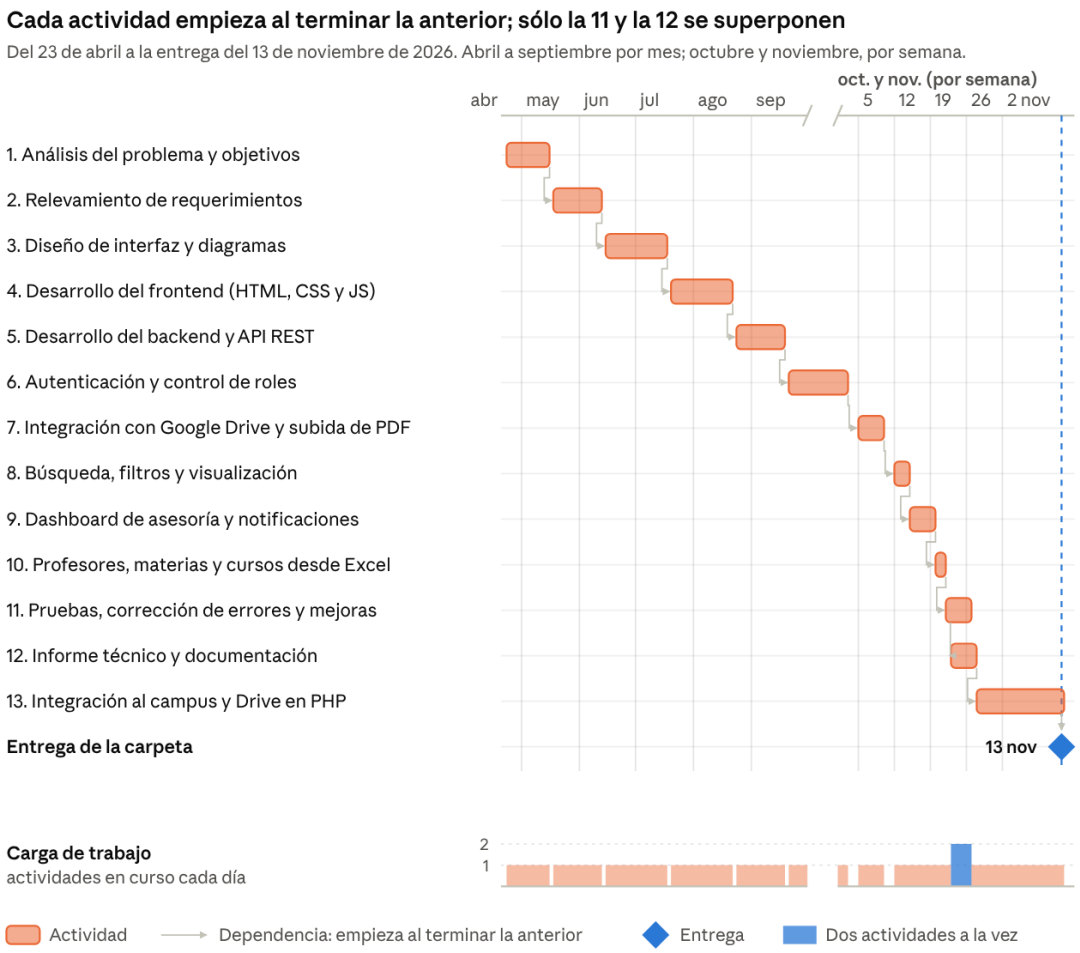


Diagrama de Gantt · 13 actividades, abril a noviembre de 2026

N.°	Actividad	Inicio	Fin
1	Análisis del problema y objetivos	23/04/2026	15/05/2026
2	Relevamiento de requerimientos	18/05/2026	12/06/2026
3	Diseño de interfaz y diagramas	15/06/2026	17/07/2026
4	Desarrollo del frontend (HTML, CSS y JS)	20/07/2026	21/08/2026
5	Desarrollo del backend y API REST	24/08/2026	18/09/2026

N.°	Actividad	Inicio	Fin
6	Autenticación y control de roles	21/09/2026	02/10/2026
7	Integración con Google Drive y subida de PDF	05/10/2026	09/10/2026
8	Búsqueda, filtros y visualización de programas	12/10/2026	14/10/2026
9	Dashboard de asesoría y notificaciones	15/10/2026	19/10/2026
10	Organización de profesores, materias y cursos desde Excel	20/10/2026	21/10/2026
11	Pruebas, corrección de errores y mejoras	22/10/2026	26/10/2026
12	Elaboración del informe técnico y documentación	23/10/2026	27/10/2026
13	Integración al campus (CampusT2), Google Drive en PHP y programa libre	28/10/2026	13/11/2026
—	Entrega de la carpeta	13/11/2026	13/11/2026

7. Casos de uso

El sistema tiene 16 casos de uso y 3 actores: alumno, docente y asesora. El docente puede hacer todo lo del alumno, y la asesora todo lo del docente para cualquier materia.



Diagrama de casos de uso · 16 casos, 3 actores

El diagrama sigue la notación UML. Las líneas llenas unen a cada actor con los casos que inicia; el triángulo hueco es la generalización (el docente hereda los casos del alumno y la asesora los del docente). Las flechas punteadas son dependencias: «include», un caso que el caso base siempre usa, y «extend», una opción que se agrega al caso base. El caso central es CU07 (resaltado).

Caso de uso	Actor que lo inicia	Relación
CU01 Buscar programas	Alumno (y por herencia, docente y asesora)	Caso base

Caso de uso	Actor que lo inicia	Relación
CU02 Ver programa en PDF	Alumno	«extend» CU01
CU03 Compartir enlace de búsqueda	Alumno	«extend» CU01
CU04 Descargar ZIP de un curso	Alumno	«extend» CU01
CU05 Iniciar sesión	Docente y asesora	Caso base
CU06 Vincular cuenta con la lista	Docente	«extend» CU05
CU07 Subir o reemplazar programa	Docente y asesora	«include» CU05
CU08 Reutilizar programa anterior	Docente y asesora	«extend» CU07
CU09 Ver versiones anteriores	Docente y asesora	«extend» CU01
CU10 Fijar fecha límite	Asesora	«extend» CU15
CU11 Corregir vinculaciones	Asesora	«extend» CU15
CU12 Eliminar programa	Asesora	«extend» CU01
CU13 Enviar recordatorios	Asesora	«extend» CU15
CU14 Pasar programas a Drive	Asesora	«extend» CU15
CU15 Ver dashboard	Asesora	Caso base
CU16 Ver registro de actividad	Asesora	«extend» CU15

Descripción de CU07: Subir o reemplazar programa

Campo	Detalle
Actores	Docente (principal), Asesora
Precondición	El docente inició sesión desde el campus (no hace falta que esté vinculado con la lista oficial).
Flujo principal	1. El docente elige curso y materia entre todas las asignaciones (las suyas aparecen primero). 2. Si es el programa para alumnos libres, marca «Es el programa libre»; el año lectivo (el actual) y el título se completan solos. 3. Elige el archivo PDF y toca **Guardar programa**. 4. El sistema verifica que sea un PDF válido de hasta 10 MB. 5. Guarda el archivo en Google Drive (Año / Curso / Materia). 6. Registra el programa, anota la acción en el registro y lo publica.
Flujo alternativo A	Ya hay un programa para esa asignación, año y tipo (común o libre): el sistema avisa, el docente confirma y el anterior pasa a Versiones anteriores.
Flujo alternativo B	Google Drive no está disponible: el PDF se guarda en el servidor y pasa a Drive automáticamente cuando vuelve.

Campo	Detalle
Excepción	El archivo no es un PDF válido, supera el tamaño o el año no es el actual: se muestra el error y no se guarda nada.
Postcondición	El programa queda publicado y la cobertura del dashboard se actualiza.

Descripción de CU06: Vincular cuenta con la lista

Campo	Detalle
Actores	Docente
Precondición	El docente inició sesión en el campus virtual y abrió el módulo de programas de examen.
Flujo principal	1. Al entrar, el sistema compara el nombre de la cuenta con la lista oficial (mismas palabras en cualquier orden, sin tildes). 2. Si hay una sola coincidencia libre, vincula la cuenta automáticamente.
Flujo alternativo	Si no hay coincidencia, el docente escribe su apellido, elige su nombre y toca **Confirmar**; queda vinculado en el momento.
Excepción	El nombre ya está tomado por otra cuenta: no se permite elegirlo.
Postcondición	Las asignaciones del docente aparecen primero al cargar y figura con su nombre de la lista; la asesora puede corregir o anular la vinculación.

8. Diagrama Entidad-Relación y diccionario de datos

La base de datos SQLite tiene cinco tablas (ProgMesas no guarda usuarios ni contraseñas: vienen del campus); la lista oficial de asignaciones vive en una planilla Excel y se relaciona con los programas por el campo `asignacion_id`.

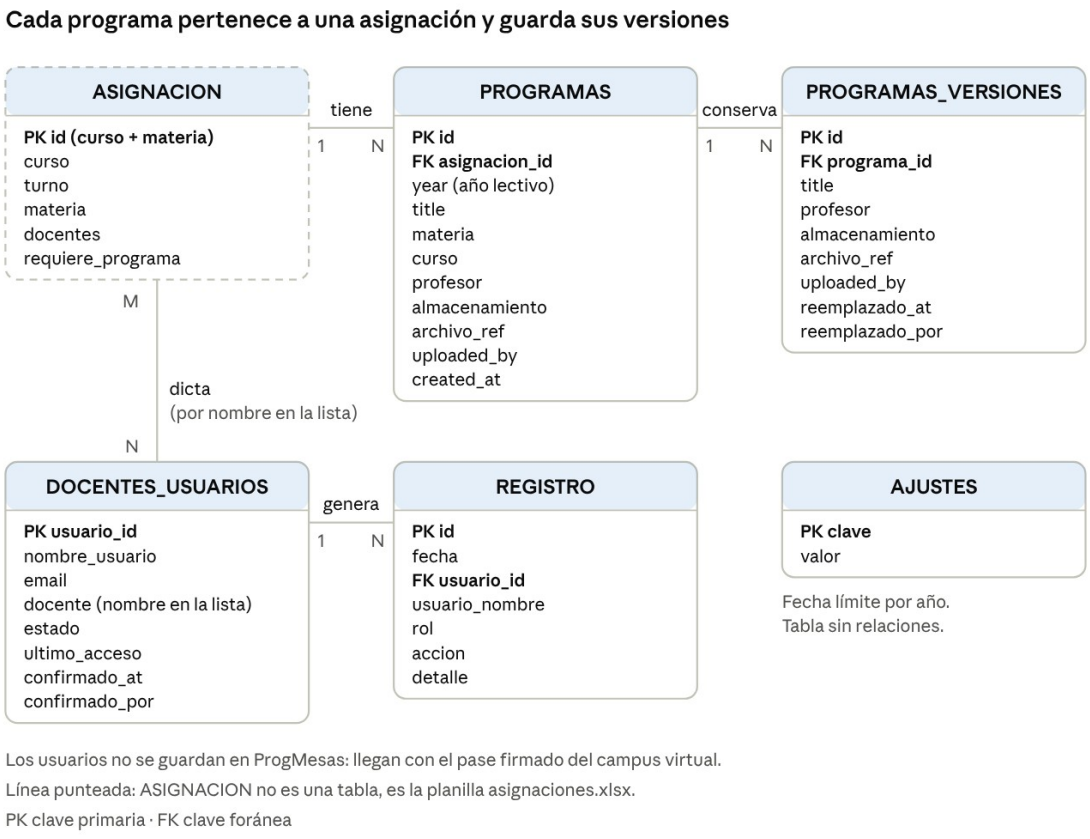


Diagrama entidad-relación · 5 tablas y la planilla de asignaciones

Cada asignación (curso + materia) tiene un programa por año lectivo y, si hace falta, un programa libre (columna tipo); al reemplazarlo, el anterior pasa a PROGRAMAS_VERSIONES. Las cuentas de docentes se vinculan con su nombre en la lista y cada acción queda en REGISTRO.

Diccionario de datos

Los tipos son los de SQLite: INTEGER (número entero) y TEXT (texto). Las fechas y horas se guardan como texto en formato ISO 8601. SQLite no declara las claves foráneas: las relaciones las controla el servidor (FK lógica).

PROGRAMAS — el programa vigente de cada asignación, año y tipo. Índice: (asignacion_id, year, tipo).

Campo	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	Número del programa (autoincremental)

Campo	Tipo	Nulo	Clave	Descripción
title	TEXT	No	—	Título automático: "Programa de Materia - Curso - Año", o "Programa libre de Materia - Curso - Año" si es el libre
materia	TEXT	No	—	Materia
year	INTEGER	No	—	Año lectivo
curso	TEXT	No	—	Curso y división, por ejemplo 2° 1°
profesor	TEXT	No	—	Docente a cargo
file_url	TEXT	No	—	Dirección para ver el PDF
fecha	TEXT	No	—	Fecha de carga (dd/mm/aaaa)
uploaded_by	TEXT	Sí	—	Cuenta del campus que lo cargó (vacío por defecto)
asignacion_id	TEXT	Sí	FK → ASIGNACIONES.ID	Curso y materia de la lista oficial
almacenamiento	TEXT	Sí	—	Dónde está el PDF: drive o local
archivo_ref	TEXT	Sí	—	Id del archivo en Drive o nombre del archivo en el servidor
created_at	TEXT	Sí	—	Fecha y hora de carga
tipo	TEXT	No	—	regular (programa común, por defecto) o libre

PROGRAMAS_VERSIONES — las versiones reemplazadas de cada programa. Índice: (programa_id).

Campo	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	Número de la versión (autoincremental)
programa_id	INTEGER	No	FK → PROGRAMAS.id	Programa al que pertenece
title	TEXT	No	—	Título que tenía
profesor	TEXT	No	—	Docente que figuraba
file_url	TEXT	No	—	Dirección del PDF de esa versión
fecha	TEXT	No	—	Fecha de carga de esa versión
uploaded_by	TEXT	No	—	Cuenta que la había cargado
almacenamiento	TEXT	No	—	drive o local

Campo	Tipo	Nulo	Clave	Descripción
archivo_ref	TEXT	No	—	Id en Drive o nombre del archivo
created_at	TEXT	Sí	—	Fecha y hora de carga de esa versión
reemplazado_at	TEXT	No	—	Cuándo se reemplazó
reemplazado_por	TEXT	No	—	Quién la reemplazó

DOCENTES_USUARIOS — la vinculación de cada cuenta de docente con su nombre en la lista oficial.

Campo	Tipo	Nulo	Clave	Descripción
usuario_id	TEXT	No	PK	Id de la cuenta del campus (rol y correo)
nombre_usuario	TEXT	No	—	Nombre de la cuenta en el campus
email	TEXT	No	—	Correo de la cuenta
docente	TEXT	Sí	FK → ASIGNACIONES.Docentes	Nombre elegido en la lista oficial
estado	TEXT	No	—	sin_vincular, pendiente o confirmado
ultimo_acceso	TEXT	Sí	—	Última vez que entró
solicitado_at	TEXT	Sí	—	Cuándo eligió su nombre
confirmado_at	TEXT	Sí	—	Cuándo quedó vinculado
confirmado_por	TEXT	Sí	—	Quién lo vinculó (automático, el docente o la asesora)

REGISTRO — cada acción que modifica datos. Índice: (fecha).

Campo	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	Número de la acción (autoincremental)
fecha	TEXT	No	—	Fecha y hora
usuario_id	TEXT	No	—	Cuenta que hizo la acción, o sistema
usuario_nombre	TEXT	No	—	Nombre de esa cuenta
rol	TEXT	No	—	Rol con el que actuó
accion	TEXT	No	—	Tipo de acción, por ejemplo programa.subido
detalle	TEXT	No	—	Datos de la acción en formato JSON

AJUSTES — configuración que cambia la asesora.

Campo	Tipo	Nulo	Clave	Descripción
clave	TEXT	No	PK	Nombre del ajuste: fecha_limite_2026 (la de cada año), drive_error y drive_sincronizado_at (estado de Google Drive en la versión PHP)
valor	TEXT	No	—	Valor del ajuste (la fecha límite AAAA-MM-DD, el último error de Drive o la hora del último paso a Drive)

ASIGNACIONES — hoja "Asignaciones" de la planilla data/asignaciones.xlsx (no es una tabla de la base).

Campo	Tipo	Nulo	Clave	Descripción
ID	Texto	No	PK	Curso y materia, por ejemplo 2-1_ingles (lo genera el sistema si falta)
Curso	Texto	No	—	Curso y división
Turno	Texto	Sí	—	Turno del curso
Materia	Texto	No	—	Materia
Docentes	Texto	Sí	—	Docentes de la materia, separados por " / "
Requiere programa	Texto	Sí	—	NO si la materia no cuenta como pendiente
Origen	Texto	Sí	—	De qué planilla de horarios salió la fila
Revisar	Texto	Sí	—	Marca del importador para controlar la fila

9. Conclusiones

ProgMesas cumple el objetivo general del proyecto: la escuela reúne, controla y publica en un solo lugar los programas de examen de sus 473 asignaciones, dentro del campus virtual que ya usan alumnos, docentes y directivos, y con los archivos guardados en el Google Drive institucional.

Logros

- Integración con el campus: cada persona entra con su cuenta de siempre y ve lo que le corresponde a su rol, sin otra contraseña.
- Carga ordenada por curso y materia, con programa común y programa libre, historial de versiones y reutilización del programa del año anterior.
- Control para la asesoría: cobertura por curso y por docente, fecha límite, pendientes exportables y registro de actividad.
- Consulta pública, enlaces por curso y descarga en ZIP, también desde el celular y en modo oscuro.
- Calidad verificada con 77 pruebas automáticas, incluida la accesibilidad (WCAG 2.1 AA) y la conexión con un Google Drive simulado.

Dificultades

- El campus lo desarrollaron otros compañeros: hubo que acordar los roles, la forma de pasar la sesión (un pase firmado) y las reglas de seguridad, y adaptar ProgMesas cada vez que el campus cambiaba.
- El servidor de la escuela corre PHP y no Node.js: la API se reescribió en PHP (`api.php`) y la conexión con Google Drive se hizo sin librerías, con la API REST de Google.
- Las planillas de horarios tenían formatos distintos y datos incompletos: el importador marca para revisar las asignaciones dudosas.
- La publicación gratuita en Render borra el disco en cada reinicio, por eso los PDF tienen que estar en Drive y la instalación definitiva es en el servidor de la escuela.

Mejoras a futuro

- Agregar al campus el rol de preceptor, que ProgMesas ya admite (consulta del dashboard sin modificar nada).
- Copias automáticas de la base de datos en la versión PHP y una tarea programada que pase a Drive lo que haya quedado en el servidor.
- Relevar la opinión de la asesoría y de los docentes con entrevistas o una encuesta después del primer período de mesas.
- Avisos por correo enviados por el sistema cuando se acerca la fecha límite.

10. Anexos

Anexo A: Relevamiento y estudio de factibilidad

No se realizaron entrevistas. El relevamiento se hizo a partir del análisis del problema, de las planillas de horarios de la escuela (de donde sale la lista oficial de 473 asignaciones) y de las definiciones que tomó el equipo durante el desarrollo:

- Los alumnos y visitantes consultan los programas sin iniciar sesión.
- Las cuentas llegan verificadas desde el campus virtual de la escuela y no necesitan aprobación. La asesoría entra al campus como directivo.
- Cada materia se controla por curso: varios docentes de una misma materia entregan programas distintos.
- Los alumnos pueden necesitar programas de años anteriores.
- Solo la asesora elimina programas y supervisa los faltantes.
- El diseño debe ser el del sistema de asistencia de la escuela (PROJECT_ASIST).

Estudio de factibilidad

Aspecto	Resultado
Análisis preliminar	Hace falta centralizar y organizar los programas de examen, y controlar cuáles faltan.
Técnica	Viable: HTML, CSS, JavaScript, PHP, SQLite y la API de Google Drive (y Node.js con Express para la versión independiente) son tecnologías conocidas por el equipo y de uso libre.
Económica	Viable: bajo costo; las herramientas son gratuitas y el almacenamiento usa el Google Drive de la escuela.
Operativa	Viable: los alumnos consultan, los docentes cargan y la asesora supervisa, sin cambiar el proceso de la escuela.
Legal	Viable: cumple las normas del marco legal (sección 4).
Social y ambiental	Reduce el uso de papel y facilita el acceso de alumnos y familias a los programas.
Conclusión	El proyecto es viable y recomendable.

Anexo B: Código

El código fuente completo está en los repositorios públicos github.com/alan4352/CampusT2 (carpeta `progmesas`, la versión que usa la escuela) y github.com/alan4352/progMesas (con el servidor Node y las pruebas automáticas). Acá se incluyen los fragmentos principales de la versión del campus: cómo el campus abre ProgMesas, cómo ProgMesas reconoce al usuario, la carga de un programa y la conexión con Google Drive.

Archivo	Qué hace
`campus/includes/progmesas.php`	Arma el pase firmado (JWT) con el usuario del campus
`campus/programas.php`	Módulo del panel: controla el permiso y abre ProgMesas con el pase
`progmesas/index.php`, `index.html`	Entrada y página principal
`progmesas/api.php`	API en PHP: rutas, permisos, base SQLite, planilla, PDF y ZIP
`progmesas/backend/drive.php`	Google Drive (API REST y OAuth 2.0)
`progmesas/backend/autorizar_drive.php`	Autoriza la cuenta de Drive de la escuela
`progmesas/frontend/`	Interfaz: listado, formulario, dashboard, estilos y modo oscuro

B.1 Campus: `includes/progmesas.php` (fragmento)

Firma el pase con la clave compartida y elige la versión de ProgMesas.

```
// true si la clave compartida ya se configuró en config.php.
function progmesasConfigurado() {
    return strlen(PROGMESAS_SECRETO) >= 32 && strpos(PROGMESAS_SECRETO,
'CAMBIAR') !== 0;
}

// Crea el pase para el usuario con sesión iniciada.
function crearPaseProgMesas($id, $nombre, $correo, $rol) {
    $cabecera = ['alg' => 'HS256', 'typ' => 'JWT'];
    $ahora = time();
    $datos = [
        'sub'   => (string) $id,
        'name'  => (string) $nombre,
        'email' => (string) $correo,
        'role'  => (string) $rol,
        'iat'   => $ahora,
        'exp'   => $ahora + PROGMESAS_HORAS * 3600,
    ];
    $json = JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES;
    $partes = base64url(json_encode($cabecera, $json)) . '.' .
base64url(json_encode($datos, $json));
    $firma = hash_hmac('sha256', $partes, PROGMESAS_SECRETO, true);
    return $partes . '.' . base64url($firma);
}

// Dirección de ProgMesas con el pase incluido.
// Si el servidor Node está activo en el puerto 3000 lo usa; si no, abre la
versión integrada (../progmesas).
function enlaceProgMesas($id, $nombre, $correo, $rol) {
    $base = '../progmesas';
    $sock = @fsockopen('127.0.0.1', 3000, $errno, $errstr, 0.12);
    if (is_resource($sock)) {
        fclose($sock);
        $base = rtrim(PROGMESAS_URL, '/');
    }
```

```
    return $base . '/?token=' . rawurlencode(crearPaseProgMesas($id,
$nombre, $correo, $rol));
}
```

B.2 Campus: `programas.php` (fragmento)

Destino de los módulos de programas de examen del panel de cada rol.

```
require_once 'includes/sesion.php';
require_once 'includes/progmesas.php';

if (!estaLogueado()) {
    $_SESSION['volver_a'] = 'programas.php'; // login.php lo usa al
ingresar
    header('Location: login.php');
    exit;
}

// Mismo control que el resto de los módulos (tabla $PERMISOS en
sesion.php)
if (!tienePermiso('programa')) {
    http_response_code(403);
    $error = 'Su tipo de usuario no tiene permiso para ver este módulo.';
} elseif (!progmesasConfigurado()) {
    $error = 'Falta configurar la clave de ProgMesas (PROGMESAS_SECRETO en
includes/config.php).';
} else {
    // El id identifica a la persona en ProgMesas: rol + correo (o DNI /
nombre si no hay correo)
    $correo = $_SESSION['correo'] ?? '';
    $id = $_SESSION['id'] ?? ($_SESSION['rol'] . ':' . ($correo !== '' ?
$correo : $_SESSION['nombre']));
    header('Location: ' . enlaceProgMesas($id, $_SESSION['nombre'],
$correo, $_SESSION['rol']));
    exit;
}
```

B.3 ProgMesas: `api.php` — quién es el usuario (fragmento)

Verifica el pase del campus o, como respaldo, la sesión del campus en el mismo navegador.

```
function verificarTokenJwt(string $token): ?array {
    $partes = explode('.', $token);
    if (count($partes) !== 3) return null;
    [$h64, $p64, $s64] = $partes;

    // Sin la clave del campus (32 caracteres o más) no se acepta ningún
pase
    if (!defined('PROGMESAS_SECRETO') || strlen(PROGMESAS_SECRETO) < 32 ||
strpos(PROGMESAS_SECRETO, 'CAMBIAR') === 0) {
        return null;
    }
    $firmaEsperada = hash_hmac('sha256', "$h64.$p64", PROGMESAS_SECRETO,
true);
    if (!hash_equals($firmaEsperada, base64UrlDecode($s64))) return null;

    $payload = json_decode(base64UrlDecode($p64), true);
    if (!is_array($payload)) return null;
    if (isset($payload['exp']) && time() >= (int)$payload['exp']) return
null;

    $id = (string)($payload['sub'] ?? '');
```

```
    $rol = mapearRol($payload['role'] ?? '');
    if ($id === '' || !$rol) return null;

    return ['id' => $id, 'nombre' => (string)($payload['name'] ?? ''),
'email' => (string)($payload['email'] ?? ''), 'role' => $rol];
}

function obtenerUsuarioActual(): ?array {
    $authHeader = $_SERVER['HTTP_AUTHORIZATION'] ??
$_SERVER['REDIRECT_HTTP_AUTHORIZATION'] ?? '';
    if (stripos($authHeader, 'Bearer ') === 0) {
        $u = verificarTokenJwt(trim(substr($authHeader, 7)));
        if ($u) return $u;
    }

    // Respaldo: la sesión del campus. La cookie viaja sola, así que un
sitio ajeno podría usarla (CSRF):
    // para modificar datos se exige la cabecera que solo manda la propia
página (frontend/common.js).
    $metodoActual = $_SERVER['REQUEST_METHOD'] ?? 'GET';
    if (!in_array($metodoActual, ['GET', 'HEAD'], true) &&
($_SERVER['HTTP_X_REQUESTED_WITH'] ?? '') !== 'progmesas') {
        return null;
    }
    if (session_status() === PHP_SESSION_NONE) {
        require_once __DIR__ . '/../campus/includes/sesion.php';   // misma
cookie segura que el campus
    }
    if (!empty($_SESSION['rol']) && !empty($_SESSION['nombre'])) {
        $rol = mapearRol($_SESSION['rol']);
        if ($rol) {
            $correo = (string)($_SESSION['correo'] ?? '');
            return [
                'id' => (string)($_SESSION['id'] ?? ($_SESSION['rol'] . ':'
. ($correo !== '' ? $correo : $_SESSION['nombre']))),
                'nombre' => (string)$_SESSION['nombre'],
                'email' => $correo,
                'role' => $rol
            ];
        }
    }
    return null;
}

function exigirRol(?array $usuario, array $rolesPermitidos): array {
    if (!$usuario) responderJson(['error' => 'Tenés que iniciar sesión'],
401);
    if (!in_array($usuario['role'], $rolesPermitidos, true)) {
        responderJson(['error' => 'No tenés permiso para esta acción'],
403);
    }
    return $usuario;
}
```

B.4 ProgMesas: `api.php` — subir un programa (fragmento)

Valida el pedido, guarda el PDF (en Drive o en el servidor), conserva la versión anterior y registra la acción.

```
/** Guarda el PDF en Drive o, si no se puede, en el servidor. */
function guardarPdf(string $contenido, array $asignacion, int $anio, string
$tipo, string $profesor): array {
    $motivo = driveFaltaConfigurar();
    if ($motivo === null) {
        try {
            $id = driveSubir($contenido, $anio, $asignacion['curso'],
$asignacion['materia'], $profesor, $tipo);
            marcarDrive(null);
            return ['almacenamiento' => 'drive', 'archivo_ref' => $id,
'file_url' => driveUrlVista($id), 'aviso' => null];
        } catch (Throwable $e) {
            $motivo = $e->getMessage();
            marcarDrive($motivo);
            $aviso = "El programa se guardó en el servidor porque Google
Drive no está disponible ($motivo).";
        }
    }
    $slug = substr(preg_replace('/\s+/', '_',
claveTexto("{$asignacion['curso']} {$asignacion['materia']} $tipo")), 0,
60) ?: 'programa';
    $nombre = round(microtime(true) * 1000) . '_' . $slug . '.pdf';
    if (file_put_contents(rutaUpload($nombre), $contenido) === false) {
        responderJson(['error' => 'No se pudo guardar el archivo en el
servidor'], 500);
    }
    return ['almacenamiento' => 'local', 'archivo_ref' => $nombre,
'file_url' => '/uploads/' . rawurlencode($nombre), 'aviso' => $aviso ??
null];
}

// POST /programas/subir
if ($metodo === 'POST' && $ruta === '/programas/subir') {
    $u = exigirRol($usuario, ['teacher', 'advisor']);
    $esAsesora = $u['role'] === 'advisor';
    $docente = $esAsesora ? '' : (docenteConfirmado($u['id']) ?:
($u['nombre'] ?: $u['id']));

    $asignacion = asignacionPorId($_POST['asignacion_id'] ?? '');
    $anio = anioDelPrograma($_POST['anio'] ?? null);
    $tipo = tipoValido($_POST['tipo'] ?? null);
    if (!$asignacion) responderJson(['error' => 'Elegí la materia y el
curso del programa'], 400);
    if (!$anio) errorAnioPrograma();
    if (!$tipo) errorTipo();

    if (empty($_FILES['pdf']) || $_FILES['pdf']['error'] !== UPLOAD_ERR_OK)
{
        responderJson(['error' => 'Falta el archivo PDF'], 400);
    }
    if ($_FILES['pdf']['size'] > MAX_PDF_MB * 1024 * 1024) {
        responderJson(['error' => 'El PDF supera el máximo de ' .
MAX_PDF_MB . ' MB'], 413);
    }
    $contenido = file_get_contents($_FILES['pdf']['tmp_name']);
    if ($contenido === false || strlen($contenido) <= 5 ||
substr($contenido, 0, 5) !== '%PDF-') {
        responderJson(['error' => 'El archivo no es un PDF válido'], 400);
    }
```

```
    $titulo = tituloAutomatico($asignacion, $anio, $tipo);
    $profesor = $esAsesora ? (implode(' / ', $asignacion['docentes']) ?:
$u['nombre']) : $docente;
    $archivo = guardarPdf($contenido, $asignacion, $anio, $tipo, (string)
$profesor);

    // Si ya hay uno del mismo tipo para ese curso, materia y año, se
reemplaza y el anterior queda en el historial
    $stmtAnt = db()->prepare('SELECT * FROM programas WHERE asignacion_id =
? AND year = ? AND tipo = ? ORDER BY created_at DESC, id DESC LIMIT 1');
    $stmtAnt->execute([$asignacion['id'], $anio, $tipo]);
    $anterior = $stmtAnt->fetch();

    if ($anterior) {
        // ... se copia el programa anterior a programas_versiones y se
actualiza la fila con el archivo nuevo
    } else {
        // ... se inserta el programa nuevo en la tabla programas
    }

    registrarAccion($u, $anterior ? 'programa.reemplazado' :
'programa.subido', [
        'id' => $id, 'curso' => $asignacion['curso'], 'materia' =>
$asignacion['materia'], 'anio' => $anio,
        'tipo' => $tipo, 'titulo' => $titulo, 'almacenamiento' =>
$archivo['almacenamiento']
    ]);
    responderJson(['ok' => true, 'id' => $id, 'reemplazado' => (bool)
$anterior,
        'almacenamiento' => $archivo['almacenamiento'], 'aviso' =>
$archivo['aviso']]);
}
```

B.5 ProgMesas: `backend/drive.php` (fragmento)

Renueva el permiso de Google y sube el PDF a Año / Curso / Materia, sin librerías: sólo la API REST de Drive.

```
/** Token de acceso vigente; lo renueva con el refresh_token cuando vence.
*/
function driveTokenAcceso(): string {
    static $cache = null;
    if ($cache && $cache['vence'] > time() + 60) return $cache['token'];

    $cfg = driveConfig();
    $falta = driveFaltaConfigurar();
    if ($falta) throw new DriveError($falta);
    $token = driveLeerJson('GOOGLE_TOKEN_JSON', $cfg['token']) ?? [];

    // expiry_date viene en milisegundos (formato de la librería de Google
que usa la versión Node)
    $vence = isset($token['expiry_date']) ? intdiv((int)
$token['expiry_date'], 1000) : 0;
    if (!empty($token['access_token']) && $vence > time() + 60) {
        $cache = ['token' => $token['access_token'], 'vence' => $vence];
        return $cache['token'];
    }
    if (empty($token['refresh_token'])) {
        throw new DriveError('El permiso de Google Drive venció. Volvé a
autorizar con: php backend/autorizar_drive.php');
```

```
    }

    $app = driveApp();
    [$estado, $cuerpo] = driveHttp('POST', $cfg['tokenUrl'], ['Content-
Type: application/x-www-form-urlencoded'], http_build_query([
        'client_id' => $app['client_id'],
        'client_secret' => $app['client_secret'] ?? '',
        'refresh_token' => $token['refresh_token'],
        'grant_type' => 'refresh_token',
    ]));
    $nuevo = json_decode($cuerpo, true);
    if ($estado !== 200 || empty($nuevo['access_token'])) {
        throw new DriveError(driveMensajeError($estado, $cuerpo));
    }
    $vence = time() + (int)($nuevo['expires_in'] ?? 3600);
    $cache = ['token' => $nuevo['access_token'], 'vence' => $vence];

    // Se guarda el token renovado (si viene de un archivo que se puede
escribir)
    if (trim((string)getenv('GOOGLE_TOKEN_JSON')) === '' &&
is_writable($cfg['token'])) {
        $token['access_token'] = $nuevo['access_token'];
        $token['expiry_date'] = $vence * 1000;
        @file_put_contents($cfg['token'], json_encode($token,
JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES));
    }
    return $cache['token'];
}

/** Busca una carpeta por nombre dentro de otra; si no existe, la crea.
Devuelve su id. */
function driveCarpeta(string $nombre, string $padre): string {
    $q = "'" . driveEscapar($padre) . "' in parents and name='" .
driveEscapar($nombre) . "'"
        . " and mimeType='application/vnd.google-apps.folder' and
trashed=false";
    $lista = driveJson('GET', '/drive/v3/files', ['q' => $q, 'fields' =>
'files(id,name)']);
    if (!empty($lista['files'][0]['id'])) return $lista['files'][0]['id'];

    $nueva = driveJson('POST', '/drive/v3/files', ['fields' => 'id'], [
        'name' => $nombre, 'mimeType' => 'application/vnd.google-
apps.folder', 'parents' => [$padre],
    ]);
    return $nueva['id'];
}

/** Sube un PDF a Año / Curso / Materia, lo deja público (sólo lectura) y
devuelve el id del archivo. */
function driveSubir(string $contenido, int $anio, string $curso, string
$materia, string $profesor, string $tipo = ''): string {
    $carpetaAnio = driveCarpeta((string)$anio, driveRaiz());
    $carpetaCurso = driveCarpeta($curso, $carpetaAnio);
    $carpetaMateria = driveCarpeta($materia, $carpetaCurso);

    $nombre = "$anio-{$curso}_{$profesor}_{$materia}" . ($tipo ===
'libre' ? '_libre' : '') . '.pdf';
    $limite = 'progmesas' . bin2hex(random_bytes(8));
    $cuerpo = "--$limite\r\nContent-Type: application/json; charset=UTF-8\
r\n\r\n"
```

```
        . json_encode(['name' => $nombre, 'parents' => [$carpetaMateria]],
JSON_UNESCAPED_UNICODE) . "\r\n"
        . "--$limite\r\nContent-Type: application/pdf\r\n\r\n" . $contenido
. "\r\n--$limite--";

    [$estado, $respuesta] = driveApi('POST', '/upload/drive/v3/files',
['uploadType' => 'multipart', 'fields' => 'id'],
        $cuerpo, "multipart/related; boundary=$limite");
    $datos = json_decode($respuesta, true);
    if ($estado < 200 || $estado >= 300 || empty($datos['id'])) {
        throw new DriveError(driveMensajeError($estado, $respuesta));
    }
    driveJson('POST', '/drive/v3/files/' . rawurlencode($datos['id']) .
'/permissions', [], ['role' => 'reader', 'type' => 'anyone']);
    return $datos['id'];
}
```

B.6 ProgMesas: `api.php` — pasar a Drive lo que quedó en el servidor (fragmento)

```
function pasarADrive(?int $limite = null): array {
    $locales = archivosLocales();
    $resultado = ['pasados' => 0, 'pendientes' => count($locales),
'errores' => []];
    if (!$locales) return $resultado;
    $falta = driveFaltaConfigurar();
    if ($falta) {
        $resultado['errores'][] = $falta;
        return $resultado;
    }
    foreach ($locales as $i => $a) {
        if ($limite !== null && $i >= $limite) break;
        try {
            $contenido = @file_get_contents(rutaUpload($a['archivo_ref']));
            if ($contenido === false) throw new RuntimeException('no se
encontró en el servidor');
            $id = driveSubir($contenido, (int)$a['year'], (string)
$a['curso'], (string)$a['materia'], (string)$a['profesor'], (string)
$a['tipo']);
            $params = [':id' => $id, ':url' => driveUrlVista($id), ':r' =>
$a['archivo_ref']];
            foreach (['programas', 'programas_versiones'] as $tabla) {
                db()->prepare("UPDATE $tabla SET almacenamiento = 'drive',
archivo_ref = :id, file_url = :url WHERE almacenamiento = 'local' AND
archivo_ref = :r")
                    ->execute($params);
            }
            @unlink(rutaUpload($a['archivo_ref']));
            $resultado['pasados']++;
            $resultado['pendientes']--;
            marcarDrive(null);
        } catch (Throwable $e) {
            $resultado['errores'][] = "{$a['archivo_ref']}: " . $e-
>getMessage();
            if ($e instanceof DriveError) {
                marcarDrive($e->getMessage());
                break; // Drive no responde: no tiene sentido seguir
probando con los demás
            }
        }
    }
```

```
    return $resultado;
}
```

Anexo C: Documentación externa (guía de uso)

Esta es la guía de uso completa de ProgMesas para cada rol. Para probar sin instalar nada está la demostración en https://alan4352.github.io/progMesas/ (en la barra amarilla de arriba, Entrar como… hace de campus; lo que se cambia queda sólo en el navegador).

Alumnos, familias y visitantes

No hace falta iniciar sesión. También se puede entrar desde el campus con **Programa de examen**.

1. Entrá a la página principal: aparecen los programas cargados, agrupados por año. Están **todos los años**, así que también se encuentran los programas de materias que se deben de años anteriores.
2. Para encontrar uno, escribí en el buscador (materia, curso, docente o título) o usá los filtros de **materia**, **curso, año y libres (Incluir libres, Sin libres o Sólo libres). Los programas para alumnos libres llevan la etiqueta Libre**.
3. Tocá **Ver PDF** para abrir el programa en el visor de la página. **Cerrar visor** lo oculta.
4. **Compartir:** lo que filtrás queda en la dirección de la página. **Copiar enlace a esta búsqueda** copia esa dirección, por ejemplo para mandar al grupo de un curso el enlace con sus programas.
5. **Descargar todos los de un curso:** elegí el curso (y, si querés, el año) y tocá **Descargar los programas de … (ZIP)**. El ZIP trae una carpeta por año; el programa libre va aparte, como «Matemática - Libre.pdf».

Docentes

La primera vez:

1. Iniciá sesión en el **campus** de la escuela con tu código de profesor y abrí **Subir los programas de examen**. ProgMesas no tiene usuario ni contraseña propios: si entrás directo a la página, **Ingresar con el campus** te lleva ahí.
2. Ya podés cargar el programa de **cualquier curso y materia**.
3. Si tu nombre en el campus coincide con el de la lista de horarios, la cuenta se vincula sola: tus materias aparecen primero y figurás con ese nombre. Si no coincide, en **Elegí tu nombre en la lista de docentes (opcional)** escribí tu apellido, elegí tu nombre tal como figura en los horarios y tocá **Confirmar**. Si no lo elegís, los programas que subas figuran con el nombre de tu cuenta del campus. Si te equivocaste, pedile a la asesora que lo corrija.

Cargar un programa:

1. En **Subir programa de examen**, elegí en **Curso y materia** el curso y la materia. Aparecen **todos** los cursos; si estás vinculado, tus materias están primero, en **Mis materias**. Cada curso lleva su propio programa: si das la misma materia en varios cursos, cargá uno por curso. Si es el programa para alumnos libres, marcá «Es el programa libre»: es un programa aparte y no reemplaza al común.
2. El **Año lectivo** es el actual y no se puede cambiar. El título se pone solo (por ejemplo, "Programa de Matemática - 2° 1° - 2026", o "Programa libre de Matemática - 2° 1° - 2026" si es el libre).
3. Elegí el **archivo PDF** (hasta 10 MB) y tocá **Guardar programa**.

Arriba del formulario aparece la **fecha límite** de entrega del año, si la asesora la fijó, y cuántos días faltan.

Usar el programa del año anterior: si para ese curso y materia (libre o no, según la casilla) hay un programa de un año anterior, al elegirlo aparece **Usar el de 2025** (o el año que corresponda). Tocalo y queda cargado para este año sin volver a subir el PDF, con el título de este año.

Reemplazar un programa: subí el nuevo PDF para el mismo curso, materia y año. La página avisa que ya hay uno cargado y, si confirmás, lo reemplaza. La versión anterior **no se borra**: queda en **Versiones anteriores**.

Versiones anteriores: en la lista, los programas que se reemplazaron muestran **Versiones anteriores (n)**. Al tocarlo aparecen las versiones viejas, con quién las reemplazó y cuándo, y se pueden abrir.

En la lista de abajo ves los programas de tus materias y los que subiste vos.

Asesora pedagógica

Entra desde el **campus** como **directivo**, con **Control de programas de examen**. Tiene todo lo de los docentes, para **cualquier** curso y materia, y además puede **Eliminar** programas desde la lista. **Dashboard**, en el menú, abre el panel de control:

- **Año** (en el encabezado, arriba a la derecha): elegí el año lectivo que querés controlar.
- **Avisos** (arriba de todo): si Google Drive no está disponible o hay programas guardados en el servidor y no en Drive. Esos programas pasan solos a Drive cuando vuelve (al abrir el dashboard, como mucho cada 6 horas); **Pasar a Drive ahora** lo hace en el momento.
- **Cobertura general:** programas requeridos, entregados, pendientes y porcentaje.
- **Fecha límite:** elegí la fecha y tocá **Guardar fecha** (**Quitar** la borra). Cada año tiene la suya. Los docentes la ven al cargar; cuando pasa, los pendientes se muestran como **Atrasados** y se cuentan los entregados **fuera de término**.
- **Notificaciones y alertas:** asignaciones para revisar en la planilla, los primeros pendientes y los **Programas sin asignación** (programas viejos que no coinciden con ninguna materia oficial): elegí a cuál corresponden y tocá **Vincular**.
- **Estado por curso:** avance de cada curso, con **Copiar enlace para alumnos** (los programas de ese curso, de todos los años) y **Descargar todo (ZIP)**.
- **Estado por docente:** avance de cada docente. A los que tienen pendientes y una cuenta con correo, **Enviar recordatorio** abre un correo ya escrito con lo que les falta y la fecha límite.
- **Programas pendientes:** lista filtrable por docente, materia y curso. **Descargar pendientes (CSV)** la baja para abrir con Excel y **Copiar correos de los docentes con pendientes** copia sus direcciones para escribirles a todos juntos.
- **Cuentas de docentes:** no hace falta aprobar a nadie, porque las cuentas ya vienen verificadas del campus. Esta sección sirve para controlar y corregir con qué nombre de la lista quedó cada uno.
    - *Entró pero no eligió su nombre:* podés escribirlo vos y tocar **Asignar**.
    - *Vinculados:* indica si se vinculó automáticamente, lo eligió el docente o lo corregiste vos. **Cambiar** corrige el nombre y **Desvincular** le quita el nombre de la lista: sus materias dejan de aparecer primero, pero puede seguir cargando programas con el nombre de su cuenta.
- **Registro de actividad:** quién subió, reemplazó, reutilizó o eliminó programas, quién cambió vinculaciones o la fecha límite, y las tareas automáticas (copias de seguridad, paso a Drive). Se puede buscar por persona, materia, curso o acción.

- **Últimos programas cargados:** los más recientes del año, con su PDF.

Preceptores

Todavía no disponible: el campus no tiene el rol de preceptor. ProgMesas ya está preparado para cuando lo tenga.

Con **Dashboard**, en el menú, ven el mismo panel que la asesora, incluido el registro de actividad, pero sin poder modificar nada (no fijan la fecha límite, no corrigen vinculaciones ni borran programas).

Menú y modo oscuro

Todas las opciones (Programas, Dashboard, Ingresar con el campus, Cerrar sesión, privacidad y condiciones) están en el **menú lateral**, como en el sistema de asistencia de la escuela. En el celular el menú se abre con el botón **Menú** de arriba a la derecha.

La página usa el modo claro u oscuro de la computadora o el celular. **Modo oscuro / Modo claro**, al final del menú, lo cambia y lo recuerda.

Preguntas frecuentes

- **No aparece mi materia en "Mis materias":** tu nombre no figura en esa materia en la lista oficial, o te vinculaste con otro nombre. Igual podés cargarla: buscala en la lista completa, debajo. Si debería ser tuya, pedile a la asesora que lo revise.
- **Me vinculé con el nombre equivocado:** la asesora lo corrige desde **Cuentas de docentes** en el dashboard.
- **"El archivo no es un PDF válido":** el archivo está dañado o no es un PDF aunque termine en .pdf. Exportalo de nuevo como PDF.
- **"Se guardó en el servidor porque Google Drive no está disponible":** el programa quedó guardado igual y pasa solo a Drive cuando vuelve. Si el aviso sigue, hay que renovar el permiso de Drive (ver Anexo D, apartado D.7).
- **Reemplacé un programa por error:** el anterior está en **Versiones anteriores**; se puede abrir y volver a subir.
- **¿Quién borró un programa?** Está en el **Registro de actividad** del dashboard.

Anexo D: Documentación interna

Documentación técnica de ProgMesas tal como funciona en la escuela, dentro del campus virtual (repositorio CampusT2, carpeta `progmesas/`). La guía de uso está en el Anexo C. La instalación de la versión Node y el historial completo de cambios están en el README y el LEEME del repositorio progMesas.

D.1 Arquitectura

CampusT2 es el sitio principal: da el inicio de sesión y el panel de cada rol. ProgMesas es una subpágina, la carpeta `progmesas/`, que usa esa misma sesión.

Modo	Qué corre	Cuándo se usa
PHP (principal)	`api.php` dentro del mismo Apache/nginx del campus	Siempre: XAMPP, servidor Debian y Render

Modo	Qué corre	Cuándo se usa
Node (opcional)	`backend/server.js` en el puerto 3000	Solo si alguien lo inicia con `iniciar.bat` / `iniciar.sh`; agrega copias de seguridad automáticas

El campus decide a cuál mandar al usuario en `enlaceProgMesas()` (`campus/includes/progmesas.php`): si hay algo escuchando en `127.0.0.1:3000` usa el servidor Node; si no, `../progmesas`. El frontend es el mismo en los dos modos: siempre llama a `api.php?r=/ruta`, y el servidor Node también acepta esa forma.

Flujo de una petición (modo PHP)

1. El navegador abre `progmesas/` → `index.php`. Si hay sesión del campus y no llegó un `?token=`, redirige a `../campus/programas.php`, que arma el pase y vuelve con el token. Si no, sirve `index.html`.
2. `frontend/common.js` guarda el token y hace las llamadas con `api()`, que agrega `Authorization: Bearer …` y `X-Requested-With: progmesas`.
3. `api.php` identifica al usuario (`obtenerUsuarioActual()`), resuelve la ruta y responde JSON.
4. Los PDF se abren desde Google Drive (vista previa pública) o, si quedaron en el servidor, como archivos estáticos de `backend/uploads/`.

Ruta	Contenido
`index.php`, `index.html`	Entrada y página principal (consulta y carga)
`api.php`	Toda la API en modo PHP: rutas, SQLite, lectura de la planilla, almacenamiento de los PDF, ZIP
`backend/drive.php`	Cliente de Google Drive en PHP (API REST v3 y OAuth 2.0, sin librerías)
`backend/autorizar_drive.php`	Autoriza la cuenta de Drive de la escuela desde la línea de comandos
`backend/oauth.json`, `backend/token.json`	Credenciales de la app de Google y permiso de la cuenta (no se suben al repositorio)
`frontend/`	`app.js` (página principal), `dashboard.html` + `dashboard.js` (asesoría), `common.js` (sesión y `api()`), `tema.js` (claro/oscuro), `styles.css`
`backend/data/asignaciones.xlsx`	Planilla oficial de cursos, materias y docentes
`backend/database.sqlite`	Base de datos (programas, vínculos, registro, ajustes)
`backend/uploads/`	PDF que quedaron en el servidor porque Drive no estaba disponible
`backend/*.js`	Servidor Node opcional y scripts para generar la planilla

D.2 Integración con el campus y autenticación

ProgMesas no guarda contraseñas. Reconoce al usuario de dos maneras, en este orden:

a) Pase JWT (principal). `campus/programas.php` llama a `crearPaseProgMesas()`, que firma con HMAC-SHA256 un JWT con `sub` (id del campus), `name`, `email`, `role`, `iat` y `exp` (`PROGMESAS_HORAS`, 8 h por defecto). La clave es `PROGMESAS_SECRETO` de `campus/includes/config.php`. `api.php` lo valida en `verificarTokenJwt()`: firma, algoritmo y vencimiento. Si la clave tiene menos de 32 caracteres o empieza con `CAMBIAR`, no acepta ningún token.

b) Sesión del campus (respaldo). Si no hay token válido, `api.php` carga `campus/includes/sesion.php` (misma cookie segura) y lee `$_SESSION['rol']`, `nombre`, `correo` e `id`. Para cualquier método distinto de GET/HEAD exige la cabecera `X-Requested-With: progmesas`, que solo manda `common.js`: así otra página no puede subir ni borrar con la sesión de alguien (CSRF).

Campus	ProgMesas	Uso
`alumno`	`student`	Consulta
`profesor`	`teacher`	Carga y reutilización
`directivo`	`advisor`	Todo, más Dashboard y eliminar
—	`viewer`	Solo lectura del Dashboard. Existe en la API, pero el campus no tiene un rol que lo genere

`/config` devuelve las direcciones de **Volver al campus** y **Cerrar sesión**, armadas con `urlCampus()` a partir de la carpeta hermana `campus/`: por eso las dos carpetas tienen que quedar una al lado de la otra.

D.3 API

Todas las rutas se llaman como `api.php?r=/ruta` y responden JSON. Los errores vuelven como `{"error": "mensaje"}` con código 400, 401 (sin sesión), 403 (sin permiso), 404, 409 o 413.

Método y ruta	Quién	Qué hace
GET `/config`	Pública	Año lectivo, límites y direcciones del campus
GET `/yo`	Pública	Usuario actual; registra el acceso del docente
GET `/programas`	Pública	Lista de programas (la asesoría ve campos extra)
GET `/fecha-limite`	Pública	Fecha límite del año
GET `/cursos/zip`	Pública	ZIP con los PDF de un curso y año, bajados de Drive o del servidor (opcional: sólo un tipo)
GET `/docentes/nombres`	teacher, advisor	Nombres de la planilla para vincular
POST `/docentes/vinculo`	teacher	El docente elige su nombre de la planilla

Método y ruta	Quién	Qué hace
GET `/programas/:id/versiones`	teacher, advisor	Versiones reemplazadas
GET `/asignaciones/mias`	teacher, advisor	Materias y cursos, con las del docente primero
POST `/programas/subir`	teacher, advisor	Sube un PDF común o libre (multipart: `asignacion_id`, `anio`, `tipo`, `pdf`)
POST `/programas/reutilizar`	teacher, advisor	Copia el programa de otro año, del mismo tipo, al actual
GET `/docentes/vinculos`	advisor, viewer	Cuentas y vínculos
POST `/docentes/vinculos/:id/confirmar`	advisor	Asigna o corrige el nombre de una cuenta
DELETE `/docentes/vinculos/:id`	advisor	Desvincula una cuenta
DELETE `/programas/:id`	advisor	Borra el programa y sus versiones, y sus PDF de Drive o del servidor si ningún otro programa los usa
PATCH `/programas/:id/asignacion`	advisor	Asocia un programa viejo a una materia de la planilla
GET `/cobertura`	advisor, viewer	Datos del Dashboard y estado de Drive
PUT `/fecha-limite`	advisor	Guarda o borra la fecha límite
GET `/registro`	advisor, viewer	Registro de actividad
GET `/programas/faltantes`	advisor, viewer	Pendientes (para el CSV)
POST `/drive/sincronizar`	advisor	Pasa a Drive los PDF que quedaron en el servidor (hasta 50 por vez)

D.4 Almacenamiento de los PDF en Google Drive

`guardarPdf()` los sube a Google Drive con `driveSubir()` (`backend/drive.php`): carpetas Año / Curso / Materia dentro de la carpeta raíz de la escuela, con el nombre `año-curso_docente_materia.pdf` (`_libre` si es el libre), y públicos de sólo lectura. En la base quedan `almacenamiento = 'drive'`, `archivo_ref` = id del archivo y `file_url` = su vista previa.

Si Drive no está configurado o falla, el PDF se guarda en `backend/uploads/` (`almacenamiento = 'local'`) y, si fue una falla, la respuesta trae un aviso. `pasarADrive()` sube después esos archivos y actualiza los programas y versiones que los usan; la llama el botón **Pasar a Drive ahora** y también `sincronizacionAutomatica()`, al abrir el dashboard, como mucho cada 6 horas.

`driveTokenAcceso()` usa el `refresh_token` de `backend/token.json` para pedir un token de acceso a Google y guarda el renovado en el mismo archivo. Si falla, el error queda en `ajustes.drive_error` y el dashboard lo muestra. El permiso se obtiene una vez con `php backend/autorizar_drive.php`.

D.5 Instalación y publicación

ProgMesas se instala junto con el campus. Requisitos de PHP: 8.1 o superior con `pdo_sqlite`, `xml`, `zlib`, `mbstring` y `openssl`, y `allow_url_fopen = On`.

Dónde	Valor	Para qué
`campus/includes/config.php`	`PROGMESAS_SECRETO`	Clave del pase (`php -r "echo bin2hex(random_bytes(32));"`)
`campus/includes/config.php`	`PROGMESAS_HORAS`	Duración del pase (8 por defecto)
`progmesas/backend/oauth.json`	Credenciales de la app de Google	Google Drive
`progmesas/backend/token.json`	Permiso de la cuenta de la escuela	Lo crea `php backend/autorizar_drive.php`
`DRIVE_CARPETA_RAIZ`	Id de la carpeta de Drive	Opcional: por defecto, la carpeta de la escuela
`GOOGLE_OAUTH_JSON`, `GOOGLE_TOKEN_JSON`	Contenido de los dos archivos anteriores	Para Render

Servidor de la escuela (Debian + nginx + PHP-FPM, `/var/www/final`)

1. Instalar `php8.4-sqlite3` y `php8.4-xml` además de lo del campus.
2. nginx no lee los `.htaccess`: usar `docs/nginx-campus.conf`, que devuelve 404 para todo `progmesas/backend/` salvo los `.pdf` de `uploads/`.
3. El usuario `www-data` necesita escribir en `progmesas/backend/` (base, `uploads/` y `token.json`).
4. Google Drive: copiar `oauth.json` a `progmesas/backend/` y ejecutar `php backend/autorizar_drive.php` desde `/var/www/final/progmesas`: abrir el enlace, aceptar con la cuenta de la escuela y pegar la dirección a la que redirige.

XAMPP: copiar CampusT2 dentro de `htdocs`, crear `config.php` y autorizar Drive con `C:\xampp\php\php.exe backend\autorizar_drive.php`. **Render:** el `Dockerfile` arma el contenedor; las credenciales de Drive van en `GOOGLE_OAUTH_JSON` y `GOOGLE_TOKEN_JSON`.

D.6 Seguridad

- **Permisos por ruta:** cada ruta que no es pública llama a `exigirRol()` al principio.
- **Pase firmado:** sin `PROGMESAS_SECRETO` válido no entra nadie por token; la clave no va en el repositorio.
- **Anti-CSRF:** con la sesión de respaldo, todo lo que no sea GET necesita `X-Requested-With: progmesas`.

- **Archivos subidos:** solo PDF (se revisa la firma `%PDF-`), máximo 10 MB, con nombre generado por el servidor.
- **Datos internos ocultos:** `.htaccess` (Apache) y `docs/nginx-campus.conf` (nginx) bloquean la base, la planilla, el código y las credenciales; `drive.php` y `autorizar_drive.php` responden 404 desde el navegador.
- **Google Drive:** `oauth.json` y `token.json` no se suben al repositorio ni a la imagen de Docker. Los PDF quedan públicos de sólo lectura, igual que en la página.
- **Auditoría:** subir, reemplazar, reutilizar, eliminar, vincular, cambiar la fecha límite y pasar archivos a Drive quedan en `registro`.

D.7 Mantenimiento

- **Planilla de asignaciones:** editar `backend/data/asignaciones.xlsx` manteniendo los `ID` y reemplazarla en el servidor; `api.php` la vuelve a leer en cada petición.
- **Copias de seguridad:** los PDF quedan en Drive; del servidor alcanza con copiar `backend/database.sqlite`, `backend/uploads/` y `backend/token.json`.
- **Actualizar:** no pisar `backend/database.sqlite`, `backend/uploads/`, `backend/data/`, `oauth.json` ni `token.json`.
- **Permiso de Drive vencido:** repetir `php backend/autorizar_drive.php`. Para que no venza cada 7 días, la app de Google tiene que estar «En producción».

Síntoma	Causa probable
401 en todas las rutas con sesión iniciada	`PROGMESAS_SECRETO` vacío, corto o con `CAMBIAR`
«No hay lista de asignaciones»	No existe `backend/data/asignaciones.xlsx`
«readonly database»	PHP no puede escribir en `backend/`
«Google Drive no está configurado»	Falta `backend/oauth.json` o `backend/token.json`
«Google Drive no está disponible: … venció»	Repetir `php backend/autorizar_drive.php`
«PHP no puede conectarse a Google»	Activar `openssl` y `allow_url_fopen` en `php.ini`

D.8 Limitaciones conocidas

Tema	Situación	Qué hacer
Rol de preceptor	ProgMesas lo admite, pero el campus no tiene ese rol	Agregarlo al campus
Copias de la base	En la versión PHP la base SQLite se copia a mano	Una tarea programada del servidor
Paso a Drive	Lo que quedó en el servidor pasa a Drive cuando la asesoría abre el dashboard o toca el botón	Si hiciera falta, una tarea programada
Base de ejemplo en el repositorio	`backend/database.sqlite` y cuatro PDF de prueba están en CampusT2	Sacarlos del repositorio antes de instalarlo en la escuela

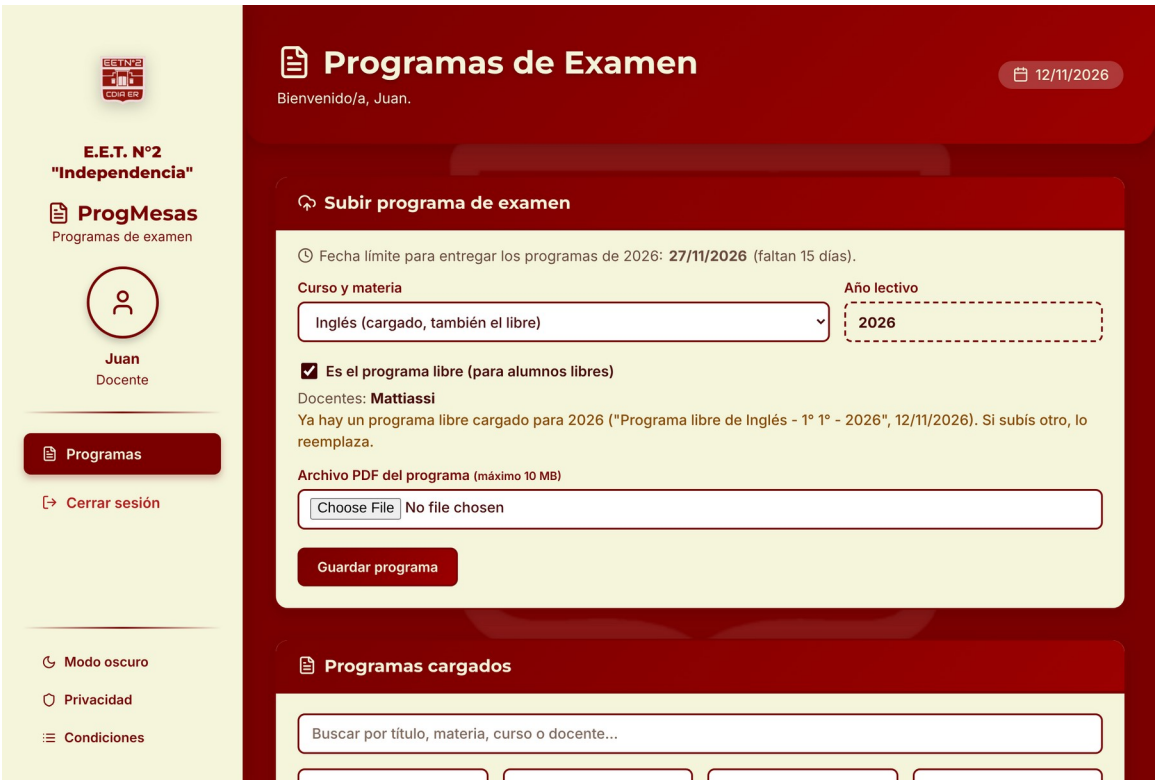
Anexo E: Capturas de pantalla

Capturas del sistema funcionando, con datos de prueba.

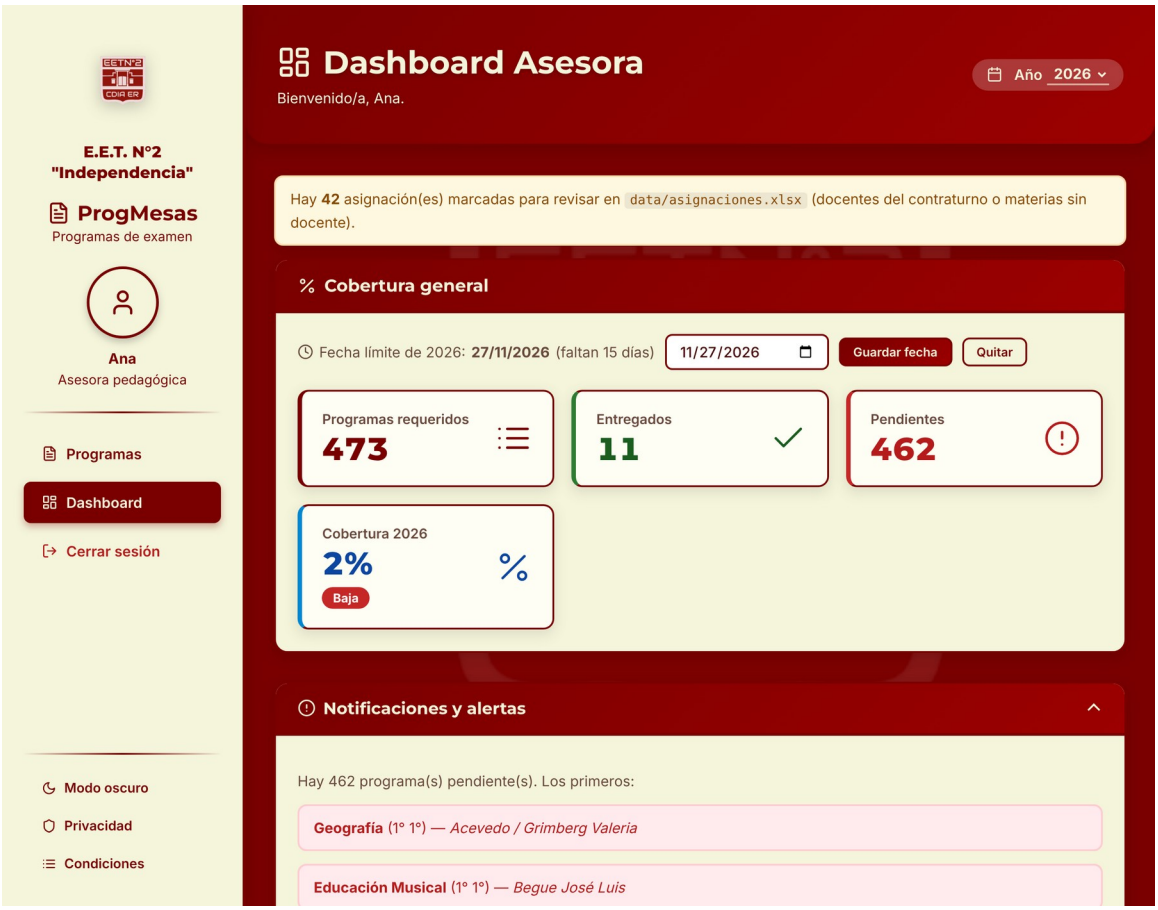
Consulta pública de los programas de un curso (sin iniciar sesión). Los programas para alumnos libres llevan la etiqueta Libre.



Carga de un programa (docente). El año lectivo es fijo, la casilla indica si es el programa libre y el sistema avisa si ya hay uno cargado.



Dashboard de la asesoría: fecha límite, cobertura del año y pendientes.



En el celular, en modo oscuro, con el enlace de un curso.


E.E.T. N°2
"Independencia"
Menú
Programas de Examen
Consultá los programas de examen de todos los cursos y años.
12/11/2026
Programas cargados
Buscar por título, materia, curso o do
Todas las materias

11. Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Buzzatto, A., y Rodriguez, V. (2026). *ProgMesas* [Código fuente]. GitHub. https://github.com/alan4352/progMesas

Google. (s.f.). *Google Drive API*. Google for Developers. https://developers.google.com/drive/api

Ley N.° 11.723. (1933). *Régimen legal de la propiedad intelectual*. Boletín Oficial de la República Argentina.

Ley N.° 25.326. (2000). *Protección de los datos personales*. Boletín Oficial de la República Argentina.

Ley N.° 26.206. (2006). *Ley de Educación Nacional*. Boletín Oficial de la República Argentina.

Ley N.° 26.653. (2010). *Acceso a la información pública: accesibilidad de la información en las páginas web*. Boletín Oficial de la República Argentina.

moshi-azz. (s.f.). *PROJECT_ASIST* [Código fuente]. GitHub. https://github.com/moshi-azz/PROJECT_ASIST

OpenJS Foundation. (s.f.-a). *Express*. https://expressjs.com/

OpenJS Foundation. (s.f.-b). *Node.js*. https://nodejs.org/

SQLite. (s.f.). *SQLite*. https://www.sqlite.org/

The PHP Group. (s.f.). *PHP: Hypertext Preprocessor*. https://www.php.net/

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/